

Orientación a Objetos II

2026

Explicación de práctica
Semana del 27 de abril



FACULTAD DE INFORMATICA



UNIVERSIDAD
NACIONAL
DE LA PLATA

Ejercicios de la semana

- Ejercicio 12: Alquiler de automóviles
- Ejercicio 13 y 13b: Monitoreo de línea de producción
- Ejercicio 14: Armado de PCs
- Ejercicio 15: Mensajero
- Ejercicio 16: Excursiones
- Ejercicio 17: Dispositivo móvil y conexiones
- Ejercicio 18 y 18b: Construcción de personajes de juegos de rol

Material teórico sobre Test Doubles



20 Abril - 26 Abril Semana actual

Esta semana se presentan dos temas relacionados con la **calidad** de nuestro trabajo como programadores: Cumplir con los requerimientos. "Que nuestro programa o módulo o clase, haga lo que tiene que hacer y no haga lo que no tiene que hacer". Entonces entender los requerimientos con exactitud y poder llegar a un consenso con el equipo de desarrollo sobre esos requerimientos es muy importante. Para esto introducimos TDD, Test Driven Development

Suele ocurrir que el programador se enfrenta con la tarea de usar objetos de los que se conoce su interface pero que no estan disponibles para agregar en el proyecto. Para esto se usa Test Double, que es un **lenguaje de patrones**



Foro - Ejercicios 8 al 11



Material sobre Test Double



Material sobre TestDrivenDevelopment



CapituloTestDouble.pdf



TestDoublePattern2026.pdf



testdoubles.mp3

Test Doubles

- Problema general
 - Es necesario realizar pruebas de un “SUT” (System Under Test) que depende de un módulo u objeto
 - El módulo u objeto requerido no se puede utilizar en el ambiente de las pruebas
 - Las pruebas pueden ejercitar
 - Configuraciones válidas del sistema
 - “Salidas indirectas” del sistema
 - Lógica del sistema
 - Protocolos
- Test Double es un **lenguaje de patrones**

Test Doubles

- Test Double
 - Crear un objeto que es una maqueta (polimórfica) del objeto o módulo requerido
 - Utilizar la maqueta según se necesite
- Rangos de implementación
 - **Cascarón vacío** → **Simulación**
 - Se generan diferentes patrones que se aplican a cada caso

Test Doubles

Patrones (**Cascarón vacío** → **Simulación**)

- **Test Stub**: cascarón vacío. Sirve para que el SUT envíe los mensajes esperados
- **Test Spy**: Test Stub + registro de mensajes recibidos
- **Mock Object**: test Stub + verification of mensajes recibidos
- **Fake Object**: imitación. Se comporta como el módulo real (protocolos, tiempos de respuesta, etc)

Ejercicio 13 - Monitoreo de línea de producción

Una empresa necesita desarrollar un sistema para monitorear su línea de producción, en la cual se usa un tanque mezclador/calentador. El tanque posee un motor que mueve paletas internas y un calentador eléctrico; con esto se puede controlar el mezclado y la temperatura del líquido contenido dentro del tanque. Además se puede controlar el vaciado (la “purga”) del tanque mediante una válvula.

Para modelar la línea de producción en este sistema se definió al proceso productivo como una secuencia de pasos y estos pasos se representaron mediante una jerarquía de clases. A continuación se detalla el esquema para dos de estos pasos:

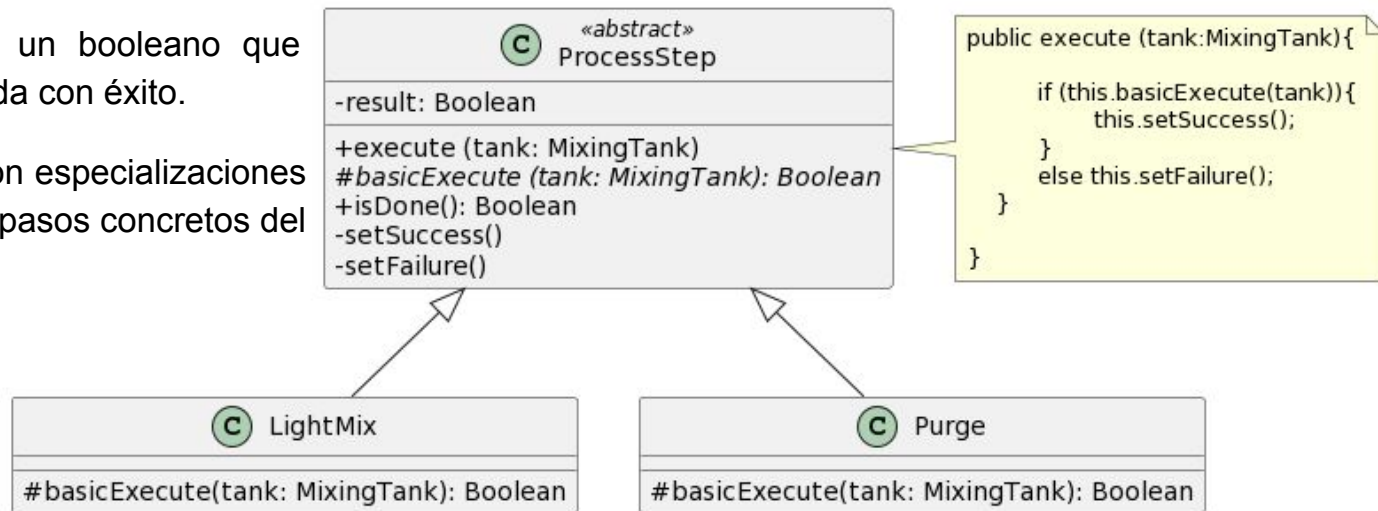
Ejercicio 13 - Monitoreo de línea de producción

La clase base ProcessStep define la estructura común para todos los pasos.

El método `execute(tank: MixingTank)` recibe como parámetro un tanque mezclador (`MixingTank`) y ejecuta sobre el tanque los comandos de la etapa correspondiente. Para esto invoca al método `basicExecute(tank)` que cada etapa implementa el cual retorna si la ejecución fue o no exitosa;

El método `isDone()` retorna un booleano que describe si la etapa fue realizada con éxito.

Las clases `LightMix` y `Purge` son especializaciones de `ProcessStep` y representan pasos concretos del proceso.



Ejercicio 13 - Monitoreo de línea de producción

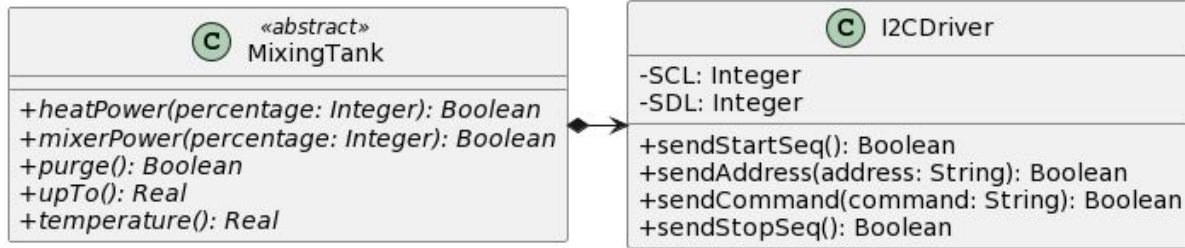
clase LightMix

```
basicExecute(tank:MixingTank){  
    return tank.heatPower(20%)  
        && tank.mixerPower(5%)  
}
```

clase Purge

```
basicExecute(tank:MixingTank){  
    return tank.heatPower(0%)  
        && tank.mixerPower(0%)  
        && tank.purge()  
}
```

Ejercicio 13 - Monitoreo de proceso industrial



No se dispone de una implementación concreta de *MixingTank* pero su comportamiento esperado es el siguiente:

- `heatPower`: configura el nivel potencia de la fuente de calor del tanque de 0 a 100
- `mixerPower`: configura el nivel de potencia de la mezcladora del tanque de 0 a 100
- `purge`: comanda al tanque para que se desagote
- `upTo`: retorna el volumen ocupado del tanque de 0 a 100
- `temperature`: retorna la temperatura del contenido del tanque

Ejercicio 13 - Monitoreo de proceso industrial

1. Implemente las clases ProcessStep, LightMix y Purge, completando el pseudocódigo provisto.
2. Implemente Test de Unidad para ambas clases cubriendo casos de éxito y falla
 - a. Explique qué tipo de TestDouble es necesario implementar para cubrir esta versión de test cases.
3. Se conocen especificaciones del tanque y se han redefinido las definiciones de LightMix y Purge. Actualice la implementación de clases y de los test cases cubriendo casos de éxito y falla.
 - a. Explique qué tipo de TestDouble es necesario implementar para cubrir esta versión de test cases.

Ejercicio 13b - Monitoreo de proceso ind. (ext)

MixingTank

tiempo en completar el método purge: 4 segundos

Transferencia de calor según el nivel de potencia recibido en heatPower (es decir, la velocidad con la que sube la temperatura del tanque mezclador depende de la potencia que se le quiere aplicar):

- 100% = + 5 °C por segundo
- 75% = + 4 °C por segundo
- 50% = + 2 °C por segundo
- 25% = + 1 °C por segundo
- 0% = + 0 °C por segundo

clase LightMix

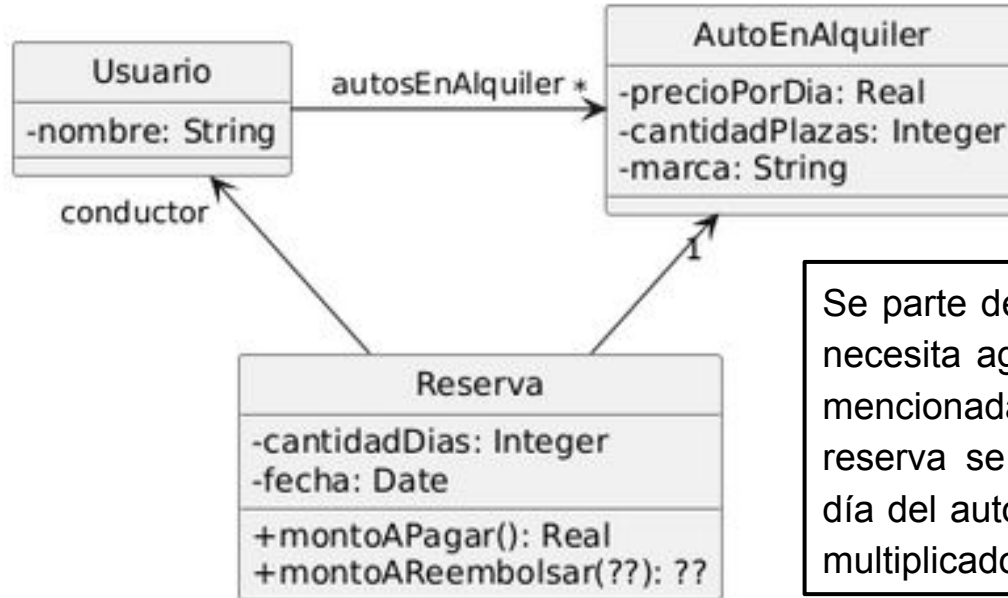
```
basicExecute(tank:MixingTank){
    temp = tank.temperature()
    tank.heatPower(100%)
    delay(2sec)
    if(tank.temperature()-temp == 10 ){
        tank.mixerPower(5%)
        return true
    }
    else {
        return false
    }
}
```

clase Purge

```
basicExecute(tank:MixingTank){
    if (tank.upTo() = 0) {
        return false
    }
    else {
        tank.heatPower(0%)
        tank.mixerPower(0%)
        tank.purge()
        delay(4sec)
        if (tank.upTo() != 0){
            return false
        }
        return true
    }
}
```

Ejercicio 12 - Alquiler de automóviles

En un sistema de alquiler de automóviles se quiere introducir funcionalidad para calcular el monto que será reembolsado (devuelto) si se cancela una reserva. Dicho reembolso podrá variar con respecto al monto total pagado, de acuerdo a la política de cancelación que sea determinada para el vehículo.



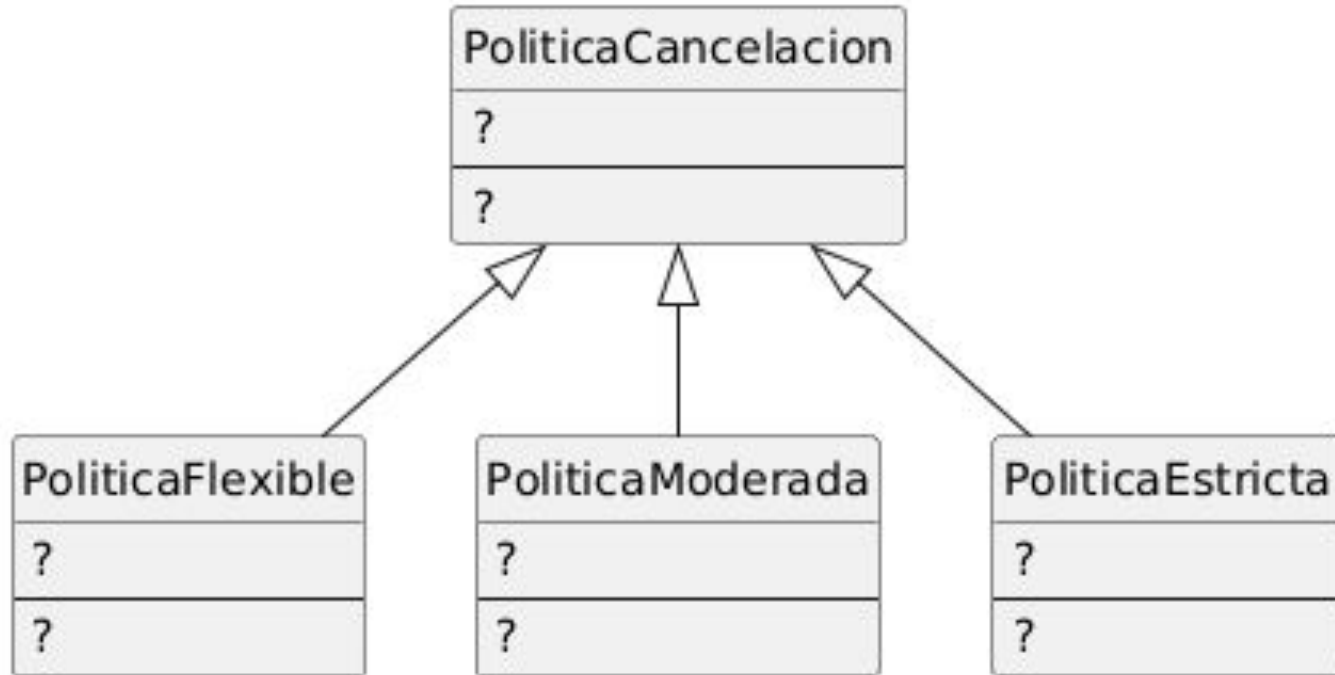
Se parte del siguiente diseño al que se necesita agregar la funcionalidad antes mencionada. El monto a pagar por una reserva se calcula como el precio por día del auto del cual se hizo la reserva, multiplicado por la cantidad de días.

Ejercicio 12 - Alquiler de automóviles

Se quiere calcular el monto a reembolsar de una reserva si se hiciera una cancelación. Dada una fecha tentativa de cancelación, se debe devolver el monto que sería reembolsado. El cálculo se hace de la siguiente manera.

- a) Si el automóvil tiene política de cancelación flexible, se reembolsará el monto total sin importar la fecha de cancelación (que de todas maneras debe ser anterior a la fecha de inicio de la reserva).
- b) Si el automóvil tiene política de cancelación moderada, se reembolsará el monto total si la cancelación se hace hasta una semana antes y 50% si se hace hasta 2 días antes.
- c) Si el automóvil tiene política de cancelación estricta, no se reembolsará nada (0, cero) sin importar la fecha tentativa de cancelación.

Ejercicio 12 - Alquiler de automóviles



Ejercicio 12 - Alquiler de automóviles

Tareas:

1. Modifique el diagrama de clases UML para considerar los cambios necesarios.
Indique el patrón de diseño utilizado y las ventajas de su uso en este diseño en particular. Documente los roles que cada clase cumple en el patrón.
2. Implemente en Java
3. Muestre en un snippet de código Java cómo crear un automóvil con una política de cancelación flexible y luego imprima en pantalla el valor de reembolso. Luego, cambie la política a cancelación moderada y vuelva a imprimir en pantalla el valor de reembolso.

Ejercicio 16 - Excursiones

Sea una aplicación que ofrece excursiones como por ejemplo “dos días en kayak bajando el Paraná”. Una excursión posee nombre, fecha de inicio, fecha de fin, punto de encuentro, costo, cupo mínimo y cupo máximo.

La aplicación ofrece las excursiones pero éstas sólo se realizan si alcanzan el cupo mínimo de inscriptos. Un usuario se inscribe a una excursión y si aún no se alcanzó el cupo mínimo, la inscripción se considera provisoria. Luego, cuando se alcanza el cupo mínimo, la inscripción se considera definitiva y podrá llevarse a cabo. Finalmente, cuando se alcanza el cupo máximo, la excursión solo registrará nuevos inscriptos en su lista de espera.

De los usuarios inscriptos, la aplicación registra su nombre, apellido y email.

Ejercicio 16 - Excursiones

Por otro lado, en todo momento la excursión ofrece información de la misma, la cual consiste en una serie de datos que varían en función de la situación.

- Si la excursión no alcanza el cupo mínimo, la información es la siguiente: nombre, costo, fechas, punto de encuentro, cantidad de usuarios faltantes para alcanzar el cupo mínimo.



Dos días en kayak bajando el Paraná

 Inicio: 15/06/2024 Fin: 16/06/2024

 Punto de encuentro: Club Náutico San Pedro

\$ 45.000

Cupo mínimo: 8
Cupo máximo: 16

 Inscriptos: 6

6 / 16

Ejercicio 16 - Excursiones

Por otro lado, en todo momento la excursión ofrece información de la misma, la cual consiste en una serie de datos que varían en función de la situación.

- Si la excursión no alcanza el cupo mínimo, la información es la siguiente: nombre, costo, fechas, punto de encuentro, cantidad de usuarios faltantes para alcanzar el cupo mínimo.
- Si la excursión alcanzó el cupo mínimo pero aún no el máximo, la información es la siguiente: nombre, costo, fechas, punto de encuentro, los mails de los usuarios inscriptos y cantidad de usuarios faltantes para alcanzar el cupo máximo.



Dos días en kayak bajando el Paraná

\$ 45.000

 Inicio: 15/06/2024 |  Fin: 16/06/2024

 Punto de encuentro: Club Náutico San Pedro

 **Usuarios inscriptos**

- juan.perez@email.com
- maria.gomez@email.com
- lucas.rodriguez@email.com
- sofia.martinez@email.com
- pablo.diaz@email.com
- carla.lopez@email.com
- martin.ramirez@email.com

 **Faltan 6 usuarios**
para alcanzar el cupo máximo

 Cupo mínimo: 8

 Cupo máximo: 16

Ejercicio 16 - Excursiones

Por otro lado, en todo momento la excursión ofrece información de la misma, la cual consiste en una serie de datos que varían en función de la situación.

- Si la excursión no alcanza el cupo mínimo, la información es la siguiente: nombre, costo, fechas, punto de encuentro, cantidad de usuarios faltantes para alcanzar el cupo mínimo.
- Si la excursión alcanzó el cupo mínimo pero aún no el máximo, la información es la siguiente: nombre, costo, fechas, punto de encuentro, los mails de los usuarios inscriptos y cantidad de usuarios faltantes para alcanzar el cupo máximo.
- Si la excursión alcanzó el cupo máximo, la información solamente incluye nombre, costo, fechas y punto de encuentro.



Dos días en kayak bajando el Paraná

\$ 45.000

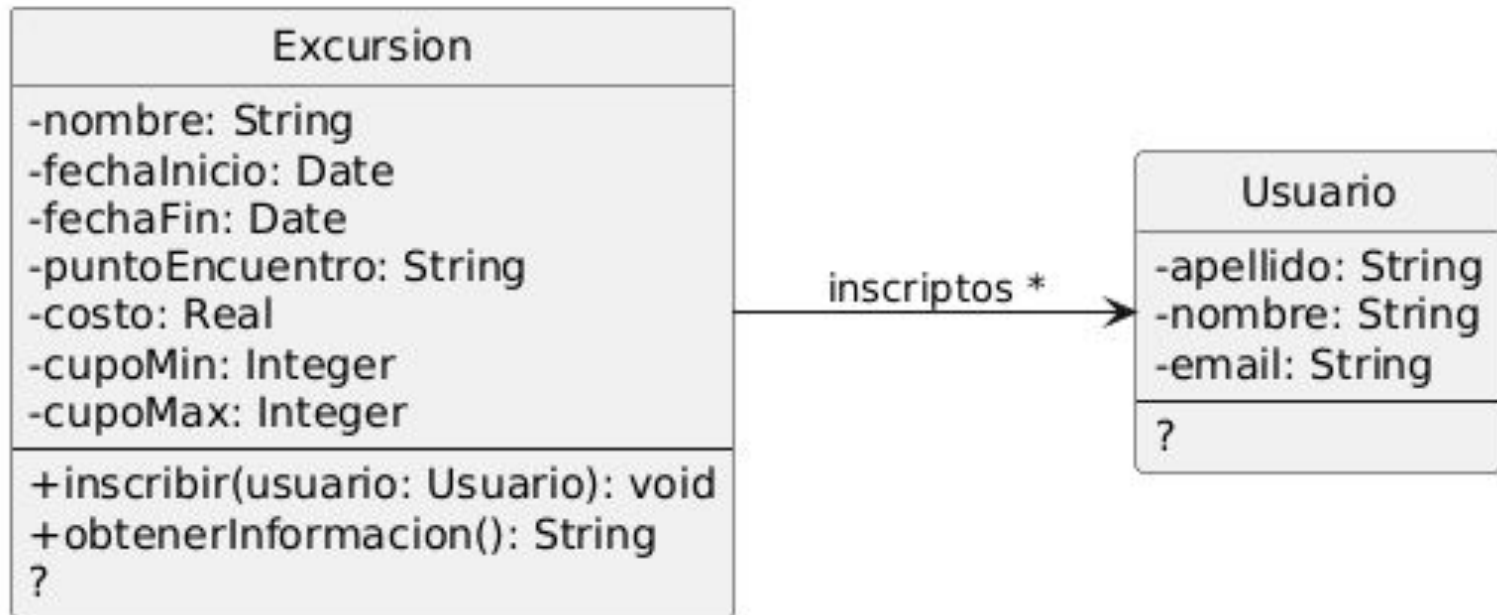


Inicio: 15/06/2024 | Fin: 16/06/2024



Punto de encuentro: Club Náutico San Pedro

Ejercicio 16 - Excursiones



Ejercicio 16 - Excursiones

Tareas:

- 1.- Realice un diseño UML. Si utiliza algún patrón indique cuál(es) y justifique su uso.
- 2.- Implemente lo necesario para instanciar una excursión y para instanciar un usuario.
- 3.- Implemente los siguientes mensajes de la clase Excursion:
 - (i) `public void inscribir (Usuario unUsuario)`
 - (ii) `public String obtenerInformacion()`.
- 4.- Escriba un test para inscribir a un usuario en la excursión “Dos días en kayak bajando el Paraná”, con cupo mínimo de 1 persona y cupo máximo 2, con dos personas ya inscriptas. Implemente todos los mensajes que considere necesarios.

¿Preguntas?